

ITPROFEL2: OPEN SOURCE PROGRAMMING

Learning Module: Finals - Week 1

Instructor: Henry V. Dela Cruz

Topic: Numeric Data Types, Datetime, and Random Modules

I. Overview and Objectives

Welcome to the Final Grading Period. We begin by mastering built-in modules that allow our programs to handle time, format dates, and generate randomized data, moving beyond simple static inputs.

Intended Learning Outcomes (ILOs):

- Review numeric data types and apply mathematical and type conversion functions.
- Import and utilize the `datetime` module to generate, format, and manipulate date objects.
- Import and utilize the `random` module to generate numbers and control randomness.

II. Numeric Data Types and Math Functions

We've discussed `int` and `float`. Python has powerful built-in mathematical functions to process these numeric types without needing external libraries.

- **Type Conversion:** Using `int()` and `float()` to explicitly cast variables.
- **Math Functions:** `abs()` for absolute values, `round()` for rounding to decimal places, `min()` and `max()` for finding extremes in a sequence.

```
# Basic Mathematical Functions
score = -85.67
print(abs(score))          # Output: 85.67
print(round(score, 1))     # Output: -85.7
print(max(10, 50, 30))     # Output: 50
```

III. The datetime Module

To work with dates and times, we must import Python's built-in `datetime` module. This is critical for logging when a user performs an action in a system.

1. Working with Date and Time Objects

```
import datetime

# Get the current date and exact time
current_time = datetime.datetime.now()
print(current_time) # Output: e.g., 2026-08-20 13:10:04.123456

# Creating a specific date object (Year, Month, Day)
foundation_day = datetime.date(2021, 9, 1)
```

2. Formatting Dates and Time

The raw `datetime` output is ugly. We use `strftime()` (String Format Time) to format it into readable text.

```
# Formatting using specific directives (%Y = 4-digit year, %m = month, %d = day)
formatted_date = current_time.strftime("%Y-%m-%d %H:%M:%S")
print(formatted_date) # Output: 2026-08-20 13:10:04
```

3. Performing Arithmetic with Dates

We use `timedelta` to add or subtract days/weeks/hours from a date object.

```
# Calculate the date 30 days from now
deadline = current_time + datetime.timedelta(days=30)
print(deadline.strftime("%Y-%m-%d"))
```

IV. The random Module

The `random` module generates pseudo-random numbers, useful for creating unique IDs, shuffling data, or picking random elements.

```
import random

# Generating a random integer between a range (inclusive)
random_id = random.randint(1000, 9999)

# Choosing a random element from a list
sections = ["BSIT-3A", "BSIT-3B", "BSIT-4A"]
assigned_section = random.choice(sections)

# Controlling Randomness (Seed)
# If you set a seed, the random numbers generated will always be the same sequence.
random.seed(42)
```

Laboratory Coding Session: ID & Timestamp Generator

Scenario: You are tasked with creating a backend script that handles user registration data. Your script must generate a randomized, time-stamped unique identifier based on user input.

Instructions (Your code must implement these exact rules):

1. Import the `datetime` and `random` modules.
2. Use `input()` to ask the user for their **Registration Year**.
3. Generate a random 4-digit number (between 1000 and 9999) using `random.randint()`.
4. Construct the **Assigned ID** by formatting the year and the random number together (e.g., 2026-8492).
5. Capture the exact current time using `datetime.datetime.now()`.
6. Format the timestamp using `strftime()` to look exactly like `YYYY-MM-DD HH:MM:SS`.
7. Output the initialized data securely to the console.

Sample Program Run (Expected Output):

```
--- SYSTEM REGISTRATION ---  
Enter Registration Year: 2026  
  
[SUCCESS] Metadata Generated.  
Assigned ID: 2026-4712  
Registration Timestamp: 2026-08-20 14:30:15
```

Submit your completed .py file to the instructor.